

IDENTIFYING AND NAVIGATING ENGINEERING PITFALLS
IN VIDEO GAME PRODUCTIONS

by

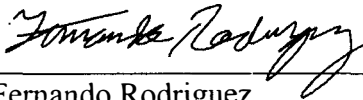
Ray Parker

A Senior Honors Thesis Submitted to the Faculty of
The University of Utah
In Partial Fulfillment of the Requirements for the
Honors Degree in Bachelor of Science

In

Computer Science

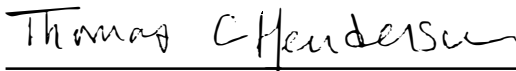
Approved:



Fernando Rodriguez
Thesis Faculty Supervisor



Mary Hall
Chair, Department of Computer Science



Thomas C. Henderson
Honors Faculty Advisor

Sylvia D. Torti, PhD
Dean, Honors College

May 2023
Copyright © 2023
All Rights Reserved

ABSTRACT

Complex team projects will experience a variety of technical issues during video game development and approaching a problem with a faulty problem-solving strategy can create a pitfall in production—every fixed bug only leads to more bugs, and developers struggle to meet their project deadlines. It is crucial to identify and resolve these issues early to prevent wasting time and funds on solutions that just do not work. This thesis revolves around the pitfalls in a yearlong video game production for my senior capstone project, which is a massive collaboration of producers, engineers, designers, and artists. As an engineer, I am responsible for coding and debugging in-game features to ensure a functional playable experience. This study will explore the engineering mishaps encountered during game development, which can be any issue that wastes time and resources, by communicating regularly with other game programmers and tracking specific problems that have caused production pitfalls. Additionally, we will identify how and when we change our problem-solving approach to address these issues and fix the production's workflow. This case study will highlight how to navigate challenges in the game development process, which can help beginner game developers overcome similar pitfalls in their own game productions.

TABLE OF CONTENTS

ABSTRACT	ii
INTRODUCTION	1
CONTEXT	3
METHODS	5
RESULTS	7
• Core Mechanic Development	7
○ Endearing Concept; Broken Execution	
○ Functioning Mechanics; Frustrating Gameplay	
• Mechanic Malfunction	14
• Lighting Developments	17
○ Performance Issues	
○ Iterative Lighting Adjustments	
DISCUSSION	24
• Playtesting	
• Revision Control	
• Collaboration	
• Iterative Development	
CONCLUSION	29
REFERENCES	30

INTRODUCTION

Game development is a massive collaborative effort that engages with several moving parts. It is a complicated process, and finishing a video game is a noteworthy feat. However, video game productions can and have ended in failure.¹ Incomplete fundraising, unorganized management, and wasted resources destine a game's development to end in an incomplete, buggy mess, or to never reach publication at all. Video game development has its moments of impulsive decisions, but no one can entirely improvise the development process. Game development is a large-scale project that needs a strict schedule and scope management to ensure its success.

Scope-management is a fundamental concern in all team projects. Scope management is “the process of identifying and defining what actions are required to deliver a project's requirements” (O'Connor). A project's scope must prioritize the most important goals and features to motivate the project towards completion while staying on budget and working to a timely schedule. In terms of game development, scope-management transforms a game from its early conceptual phases into an actual playable product. By consistently referencing scope throughout game development, developers stay on a consistent schedule and avoid scope creep² by carefully managing low-priority tasks that may stall the game's production.

Game developers must recognize when their tasks harm the momentum of game development. With the wrong focus or approach, a hard-working developer can still be

¹ Several video games have had poor reception due to terrible marketing or frustrating game quality. *Cyberpunk 2077* and *Duke Nukem Forever* are some of the games infamous for their buggy releases, which harmed their marketing and resulting revenue (Draven).

² Scope creep is the concept of a project expanding beyond its intended scope, which leads to “missed deadlines, blown budgets, and the delivery of a finished project that does not match the outlined purpose of the project charter” (O'Connor).

wasting time and be working outside of the game's scope. Spending time on a secondary feature of the game, like adding cool hat selections for a character, instead of ironing out game-breaking bugs impedes the game development process. Developers cannot spend too much time on low-priority, faulty, or expensive approaches to their games without risking the time and resources needed on completing the game production.

I reference these situations as **pitfalls**: problematic approaches that sneak up on a developer and trap them in a cycle of wasting time and resources. This harms the development process when their project has a strict scope and schedule to upkeep. Pitfalls are a direct danger to a project's scope. Thus, a developer cannot be stuck in a pitfall for too long; it is crucial to recognize when they are trapped in one and learn how to escape it to optimize the production of their game.

My work focuses on these game development pitfalls—I searched for common signs of pitfalls that occurred in Whisker Witchery, a student game completed for a Capstone project, and analyzed how developers can learn to navigate them. By learning to recognize game development obstacles and recognizing them early, developers can minimize the damage of wasted resources and optimize the time spent on game development. This research can ideally teach upcoming game development professionals how to identify pitfalls in their own game productions and learn how to navigate through them.

CONTEXT

The University of Utah's Entertainment Arts and Engineering program (EAE) provides video game education and hands-on game development experiences to their students.³ Capstone is a two-semester course that engages students in creating a new video game from start to finish. Some of their opportunities include ideating and refining game ideas, prototyping basic levels, implementing new mechanics and features, playtesting and removing bugs, game marketing, and more. Students bring their own skills and opportunities to their teams and fully develop an idea into their own video game experience.

Whisker Witchery (Figure 1) is a student-led video game project, and it was developed through the Entertainment Arts and Engineering program during the Fall 2022 and Spring 2023 semesters. This game is a 3D stealth-puzzle platformer implemented in Unreal Engine 5. In this game, a jealous witch curses their rival – the main character – by turning them into a rat, and the player explores the witch's cottage to collect ingredients to make a potion to become human again. They use potions to bring plants to life, scurry up and down vines, and hide under a magical witch's hat to avoid enemies.

The game uses stealth-oriented puzzles, thoughtfully crafted platforming, and a fantastical 3D environment (Figure 1) to craft its experience. Each level challenges the player to overcome a unique platforming and stealth challenge, such as leaping across flying books or dodging fireball-spitting candles, and they collect a potion ingredient from a treasure chest by the end. The player is immersed into this witch-savvy cottage decorated with informative scrolls, jars of eyeballs, loose skulls and mandibles, and a sparkling

³ EAE is a top-ranked video game development program at the University of Utah. More information is listed on their official page: <https://games.utah.edu/>

cauldron used as the game's centerpiece. The game intends to immerse the player into a spellbound world from the perspective of a tiny critter.



Figure 1: Whisker Witchery video game box cover⁴ and environmental screenshots⁵.

As one of the software developers on Whisker Witchery, I identified software-oriented pitfalls from our personal game development experience. My conducted research involves identifying these pitfalls, spotting their warning signs, and analyzing our strategies for escaping pitfalls from all stages of development.

⁴ The Whisker Witchery video game box cover was created by Elena Southworth, one of the main 2D illustrators on our project.

⁵ The 3D models, textures, and design contributing to the level is a collaborative work by our game development team. The images' composition and high-resolution screenshots were created by Morgan Hanephin, one of Whisker Witchery's designers.

METHODS

For my methods, I journalled our engineering-specific obstacles over eight months of production, which ranges from our very first prototype to the early-access game demo⁶, and documented how our development team navigated these issues. We dealt with our fair share of obstacles during development, and I recorded these in a personal digital document (Figure 2) to keep track of them.

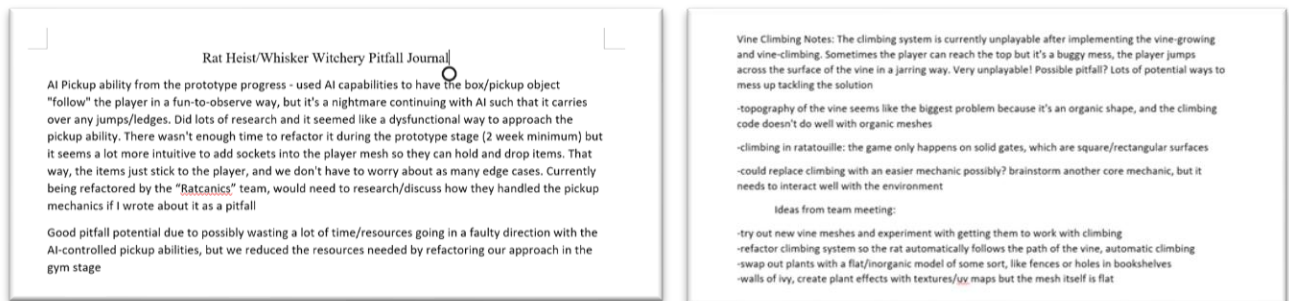


Figure 2: A visual recreation of the journal used to sketch out thoughts and track ideas

By journalling, I had an easy-to-access resource for sketching out my thoughts and observations during Whisker Witchery's development. Since every game production is different with their unique game ideas, developers, issues, and culture, trying to predict the pitfalls would be wasted effort. Instead, my goal was to observe, reflect, and learn about our pitfalls naturally over the game's development, so I stayed attentive to our engineering obstacles and journalled these observations. This included software bugs mentioned in team meetings, stubborn issues that carried over into playtesting feedback, and personal frustrations about the game's software design. Frustration could indicate feeling trapped or

⁶ Whisker Witchery's prototype development began on September 9th, 2022, and the early-access game released on Steam on April 20th, 2023.

stuck, thereby hinting at the presence of a pitfall. I focused on recording loose thoughts and observations and revisited them often to see if I discovered any useful pitfall developments.

If the recorded issues had simple fixes without much overhead, I marked them as regular bugs instead of pitfalls. However, whenever the problem appeared regularly – whenever they trapped us in a cycle of wasting time and resources, or they threatened to trap us in such a cycle – I recognized these as our production pitfalls unique to our team and regularly journalled their development. I wrote down the warning signs I noticed, our brainstormed solutions, any successful or failed attempts at escaping the pitfall, and other points of interest that emerged during the pitfall's development. Additionally, if I connected game development principles to the pitfall, I conducted some light research to review these principles and saved the sources in my journal appropriately.

As we neared the end of the semester and planned to publish our game, I picked some of the most lesson-oriented pitfalls from my journalling experience and analyzed the results in-depth. I identified the signs of our team being trapped in an engineering pitfall, how we chose to escape this pitfall, and how our approaches contributed to our game's production. To apply these findings to other game development projects, I generalized my findings into key takeaways as well, and I supported these findings with additional literature reviews.

Thus, we will dive into some of Whisker Witchery's pitfalls and analyze our approaches and strategies for handling them.

RESULTS

By journaling the obstacles from Whisker Witchery's development, I analyzed how we recognized and approached our engineering-specific pitfalls. Each section highlights a detailed instance from Whisker Witchery and investigates their related pitfalls, including the identifiers of hitting a pitfall and our approach to escaping the situation. These pitfalls, described in subsections, end with a key takeaway from the experience such that game developers can apply similar principles to their game productions.

Core Mechanic Development

Core mechanics are “an action or set of actions that players will repeat over and over as they move through the designed system of a game” (Zimmerman). If a game's core mechanics execute well and are entertaining to interact with, it can create a fun and cohesive game experience for the player.

In Whisker Witchery, climbing had been one of our core mechanics since its earliest stages. Although our developers conceptually loved the climbing mechanic, we encountered several engineering mishaps during production. This section outlines the various pitfalls encountered with our climbing mechanic and how we solved them.

Endearing Concept; Broken Execution

When we were brainstorming mechanics in our early development stages, we thought climbable surfaces contributed to a fun platforming experience while also leaning into the rat nature of the character. We conceptualized the core mechanic with a 2D sketch

(Figure 3), which visualizes the player growing a potted vine for climbing and reaching a goal.

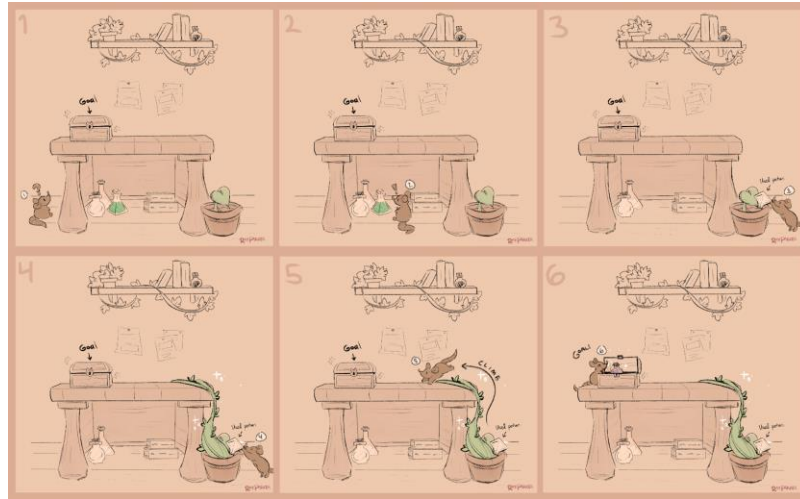


Figure 3: Whisker Witchery core mechanic and loop example⁷

Unfortunately, after implementing the concept in-engine (Figure 4), the climbing mechanic was impossible to use. We successfully implemented the vine-growing and climbing mechanics, but whenever the player attempted climbing the vine, their character would fly off the vines and almost never reached the top. The topography of our vine model did not create a smooth vine-climbing experience.

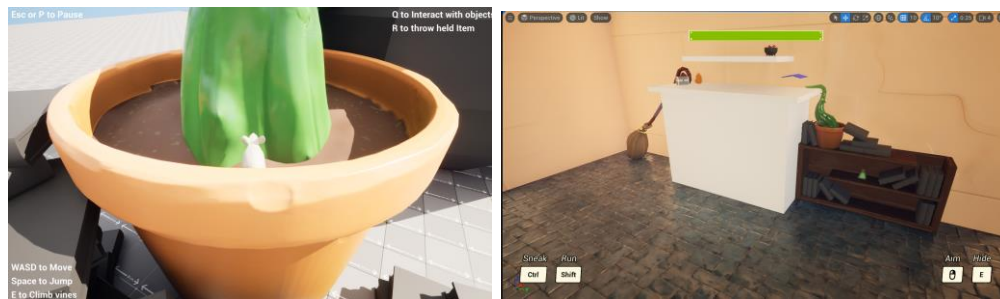


Figure 4: Prototype implementation of vine-climbing mechanic⁸

⁷ The core mechanic conceptualization and visual guide was created by the thesis author, who is a 2D artist and software developer on Whisker Witchery.

⁸ Screenshot and design layout provided by Morgan Hanephin, one of designers on Whisker Witchery.

With a broken core mechanic, we immediately identified this issue as an engineering pitfall. When considering potential solutions, most of them would create a similarly low-quality mechanic. If we redesigned the topography of the vine, we needed smooth, climbable surfaces that went against the organic nature of vines, which would be difficult to design. If we reinvented climbing such that the player automatically scaled the vine and removed room for error, this creates a less-appealing mechanic using expensive code-restructuring. We had to cautiously choose our solution such that it solved our issues without wasting too many resources: one of our earliest pitfalls in production.

Ultimately, we escaped this pitfall by redesigning the vines in an intuitive way. Vine stalks have such a curvy, organic shape that worked poorly with our existing mechanic, so we opted for redesigning the plant such that it could be represented with a simpler topography: walls of overgrown vines. These walls could be represented in-game as inorganic, flat surfaces that meshed better with our climbing mechanic.

This solution (Figure 5) reduced tons of overhead to our climbing and greatly benefited the game's scope. Our 3D artists only needed to create tiles of overlapping vines instead of multiple pots of vine stalks, saving time and energy on their workloads. This new design worked much better with our surface-centric climbing code, which allowed the player to climb and reach intended destinations. Overgrown vines had better flexibility and adjustment as well, because they could be scaled differently to any vertical platform. Thus, this pivot solved our first climbing pitfall in a quick and efficient manner.

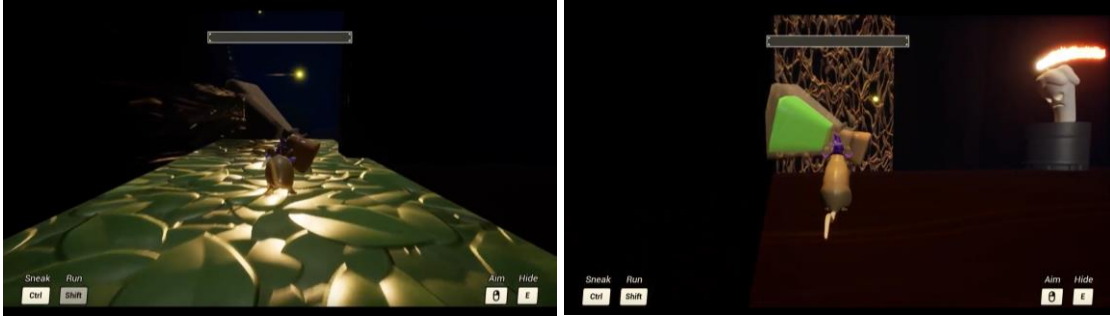


Figure 5: Growing walls of overgrown vines with a vine-regenerating potion⁹

This is a golden example of identifying and solving an engineering pitfall. An almost-unplayable mechanic is an obvious warning sign that something needs to change. Our solution reduced overhead for all the developers working on the climbing mechanics, which reduced the time and resources needed to create our climbing function. By identifying pitfalls quickly and solving them mindfully, we solved several problems at once while reducing the resources needed for the solution.

The takeaway of this pitfall is to implement all concepts into a game engine as soon as possible. We illustrated a fun climbing mechanic, but the actual implementation led to unprecedented, game-breaking issues that could not be predicted in the conceptual stage. Unpredicted issues are expected in any software development project¹⁰, including game development, so engineering our concepts as soon as possible provides enough time to locate and navigate these issues. Game developers should implement their ideas into a game engine as soon as possible to allow enough time to revise unexpected pitfalls.

⁹ The vine-wall design was created and textured by Jordan Tolbert, one of our 3D artists, and the vine-growing potions and climbing mechanics were engineered by Grant Chang and Ray Parker.

¹⁰ Video games, like all other software applications, are susceptible to bugs and issues in their software. Debugging is a valuable aspect of the software development cycle to find unexpected issues and improve their functionality.

Pitfalls are not always game-breaking and thus easy to identify, and the first solution is not always this successful. As we continued developing our climbing mechanics, a more complex pitfall appeared that required more time to solve it.

Functioning Mechanics; Frustrating Gameplay

Our revamped climbing mechanic technically worked, but the gameplay itself had a myriad of bugs and issues. Designers struggled to implement overgrown vines into their levels because of the delicate nature of the existing code. Leaving a checkbox unchecked or slightly misplacing the climbing object led to frustrated designers with unclimbable surfaces. Additionally, the climbing was disorienting, and the camera struggled when transitioning between walking and climbing (Figure 6). Despite climbing being one of our game’s most interesting features, our playtesters found this mechanic to be more frustrating than fun.

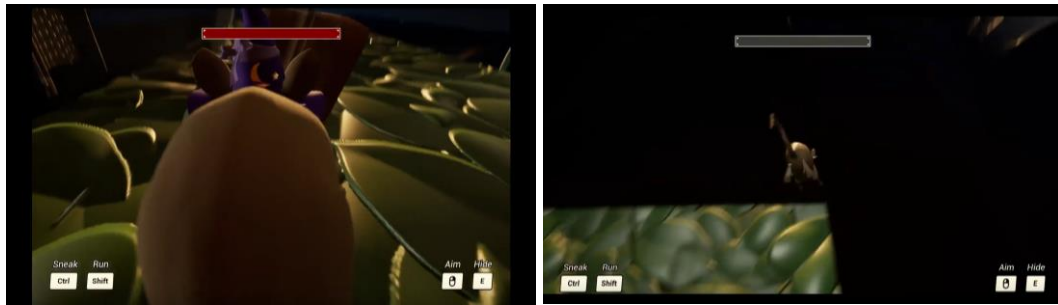


Figure 6: Examples of awkward camera situations during climbing sequences

This pitfall impeded our development in a less observable way than the last one. After our climbing mechanic functioned, our team had higher priorities to focus on, so this issue stuck around and weakened our gameplay for several months. However, as we

entered the second semester of our capstone and our game became feature-complete¹¹, the climbing mechanic gained higher priority and we needed to pivot our approach yet again.

We identified this pitfall and raised its priority after receiving enough playtesting feedback. During most of our development, we had firm playtesting feedback multiple times a month, and playtesters frequently criticized the awkward flow of the climbing and its disorienting camera movement. Additionally, our designers kept struggling with implementing the climbing mechanic into levels, and this created frequent bugs for the engineers to investigate.

Initially, our pitfall approach involved small quality-of-life solutions, such as UI hints for how to grow vines and tinkering with the camera settings. These were useful, but as we continued to receive negative feedback despite these changes, we realized we fell into a larger pitfall than we initially thought. Fixing the current climbing mechanic seemed to create more bugs, and the climbing continued to feel rough and disorienting. Our minor fixes did not address the root of the problem, so we stayed stuck in this pitfall for some time.

Our team finally remedied the situation by remaking the climbing system's codebase. Previously, we referenced Unreal game engine tutorials for climbing mechanics, but these systems often landed us in a similar buggy state—thus, we deviated from the tutorials and created our own climbing structure. We analyzed intuitive climbing mechanics in other games (such as *Breath of the Wild*), drafted a design document, and

¹¹ During a game production, being “feature-complete” means developers were prohibited from creating new game mechanics. This encourages developers to polish their existing mechanics and focus on bug-fixing instead.

completely refactored the vine-growing and climbing systems. This was an expensive approach in terms of time and resources, but our producers deemed it worthwhile when smaller fixes seemed to waste resources instead.

By refactoring the climbing code entirely, the developers could consciously choose at each step how they wanted to re-implement climbing. Starting from scratch is an expensive situation for time and resources, but it provides additional clarity during development, allows more freedom for system changes, and dismisses any previous bugs that may have been difficult to locate.

The finalized, refactored climbing code still bared similarities to its previous system, but we rehandled the camera system, condensed the vine-growing code, and significantly changed the visuals (Figure 7). Playtesters loved the feel of the new climbing, designers could easily implement vines into their levels, and their revamped visuals blended much better into the environment. We eliminated the jarring feeling of the climbing mechanic and playtesters found it fun to play again. Our approach eliminated the problems of the pitfall and enhanced our game's experience.



Figure 7: Refactored climbing system with improved visuals¹²

¹² The improved textures and visuals were vastly done by Jordan Tolbert. The refactored climbing system codebase was designed and implemented by Grant Chang.

The takeaway for this pitfall is to playtest during game development and to playtest constantly. Instead of impeding game development directly, some pitfalls harm the game's experience instead, and frequent playtesting illustrates these less-obvious issues to the developers¹³. By playtesting in the earlier stages of game development, developers can identify these complex issues early enough in the process to have the bandwidth for expensive solutions. Frequent playtesting is a fantastic way of locating obscure pitfalls and allotting enough time to escape them.

Mechanic Malfunction

Game development has its highs and lows, and the lows can strike a development team at any point of production. Long-lasting mechanics may break, playtesting can reveal twice as many game-breaking issues as before, and bugs may appear from the most bizarre sources. In the face of any sudden pitfalls, game developers need to be flexible in anticipating and handling them.

During Whisker Witchery's final month of development, we endured a frightening 48-hour period where unrelated pieces of code ceased to work. Several core mechanics malfunctioned at the same time, and when core mechanics are the most essential component for gameplay, this terrified our team so close to the game's release. This section navigates the fear of this type of pitfall and how our development team overcame it.

¹³ Playtesting is a crucial part of quality control for game development. There are several types of testing such as concept, stress, and accessibility testing to reveal unexpected issues to the developers so they can improve the gameplay and the user's experience (Dave).

The pitfall began as a small bug that intensified into several game-breaking obstacles. While one of our developers worked on level design, they discovered that one of the mechanics, a glass bottle used for distracting AI enemies, had stopped working entirely (Figure 8). As they investigated this issue, they discovered that the AI enemies stopped moving entirely in one section of the game. While attempting to fix this, the AI enemies stopped functioning in the rest of the game as well. In this manner, several bugs would appear at random and worsened the situation. Some objects lost gravity when thrown by the player; checkpoints no longer worked; the player stopped respawning entirely. The game only seemed to break more as developers jumped in-engine to fix the earlier issues.

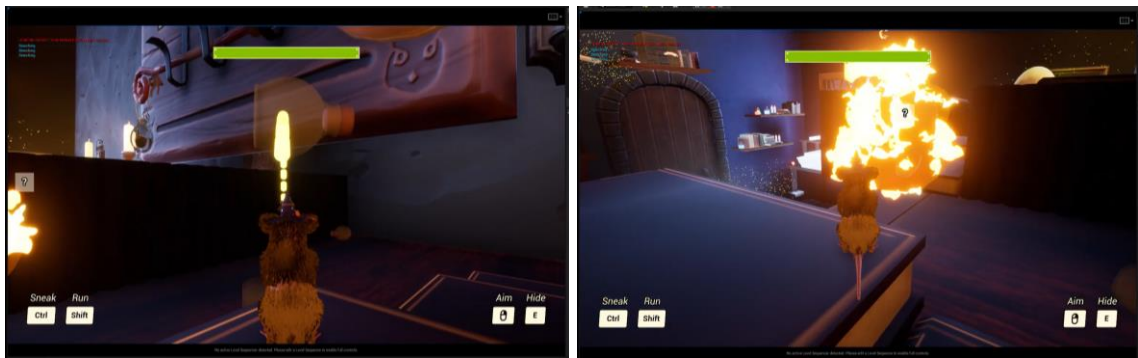


Figure 8: An example of the AI enemies failing to chase after intended distractions

This is a daunting type of pitfall to encounter. This goes beyond just one mechanic breaking—multiple mechanics had malfunctioned, and it compromised the playability of the game. We identified the pitfall through the sheer volume of game-breaking mechanics, which the designers communicated quickly to the engineers to prevent the issue from escalating further.

By utilizing our revision control system¹⁴, our team was capable of rationally approaching and fixing the situation. Our revision control system allowed us to navigate through the project's history, which helped us narrow down the files that changed and discover when the game started breaking. This significantly reduced the work needed to locate the source of the game-breaking bugs. Additionally, Perforce¹⁵ allows developers to jump between previous versions of the project, which helps with diagnosing the specific version that led to creating certain bugs. Thus, revision control allowed our developers to quickly navigate the project's upload history until they found the game-breaking updates.

Surprisingly, none of the mechanics had been tampered with—the only change happened within the game map¹⁶ itself. A change in the map itself somehow broke several mechanics across the game. The mechanics still functioned in other game maps normally, which was relieving, but it was also frustrating because locating the game map's bug-creating changes was an arduous task. Instead of narrowing down our issue to a few lines of code, we had thousands of objects in a massive 3D space with their own individual settings, and any of these factors may have caused the malfunction. Our developers narrowed down the issue, but, after spending hours in the level tinkering with different items and settings, the issue was far from resolved.

Ultimately, we escaped the pitfall by rolling back to a previous version of the game's map. Due to the sheer size of the level, developers wasted hours on adjusting

¹⁴ Revision control, also known as source control and version control, is a system that allows multiple users to backup files, integrate their work with one another, and roll back to previous versions of the project as needed (Ernst).

¹⁵ Perforce Software is a revision control system used in Whisker Witchery's production.

¹⁶ In this context, a game map is the physical environment of the game that developers can work on and interact with. This includes placing new game assets or designing levels.

random objects or changing instance settings to stumble upon the issue. However, since we used revision control, our developers were able to revert the game map to a previous version that still functioned. This solved all the sudden bugs and, by saving a copy of the broken map before reverting it, the original designer could copy over their level design progress instead of restarting from scratch. By the end, the mechanics all worked as expected again without deleting the level design progress.

The key takeaway is that game developers need revision control on their projects to save themselves from any developmental nightmares. Even if a series of issues is created by a single change, locating and fixing the issues within a reasonable amount of time is not always possible. Revision control not only expedites the process of locating when and how bugs occur but also provides the option of reverting to previous code, which can potentially save several resources depending on the nature of the bugs. All game development projects need revision control to properly deal with sudden, intensifying issues within a reasonable amount of time.

Lighting Developments

During Whisker Witchery's development, our team implemented several lighting effects to truly immerse the player into a witch-worn world. Lighting, however, is complicated in game engines, and we faced numerous challenges and pitfalls with properly implementing them into the game.

Performance Issues

To enhance the atmosphere in our game, we allocated several resources to our lighting production. Sparkly particles drift throughout the level, the potions give off a cool, smooth glow, candles light the way to the player's next goal, and moonlight splashes across the level from a large and open window (Figure 9). The level's props and design already leaned into the witchy aesthetic, but the extra details from our lighting helped our game feel magical. Our lighting developments became integral to our game and magnified the player's experience.



Figure 9: Early lighting implementation into Whisker Witchery¹⁷

Unfortunately, the game's performance plummeted shortly after our lighting updates, and the game's framerate dropped to around 5-10 FPS¹⁸. This is terrible for a platformer game, because the jarring nature of such a low FPS causes an almost unplayable experience where the players struggle to jump across even the easiest platforms. After all the hard work and payoff from the magical lighting, reverting changes felt like a

¹⁷ The lighting experimentation and implementation was mostly done by our developers Morgan Hanephin and Jordan Tolbert.

¹⁸ FPS stands for Frames Per Second, which is "the number of images consecutively displayed each second" (Rouse). In gaming terms, 30-60 FPS is the ideal frame rate to ensure a baseline playable experience (Klappenbach).

disheartening decision, so we needed to devise a solution that fixed the performance without compromising the lighting updates.

We identified this issue as a pitfall due to the sheer struggle to solve it. Our lighting team attempted several optimizations to fix the performance, which included experimenting with material instancing, texture sizes, Unreal Engine 5's Nanite¹⁹, light complexity, level-of-detail biases, and volumetric fog pixel size²⁰. This led to some optimization improvements and improved our FPS, but we still fell under the preferred 30-60 FPS framerate. Our game continued lagging and frames seemed to cut out, leading to choppy movement and painful platforming. Thus, since our time and resources failed to yield the results we wanted, and the poor framerate prevented other developers from testing other aspects of the game, our lighting issues created another game development pitfall.

We escaped the pitfall by communicating the game-breaking issue to the rest of our team and combining our expertise to collaborate on a solution. The producers, engineers, and other sub-teams on Whisker Witchery were unaware of the lighting issues until the lighting developers asked for additional help from the rest of the team. Lighting became the focus of our following team meeting, which encouraged most of our team to playtest the game and report back their personal experiences with the performance issues. By playtesting on several computers and sharing computer specifications, we discovered that

¹⁹ Nanite is a “virtualized geometry system which uses a new internal mesh format and rendering technology to render pixel scale detail and high object counts” (Unreal Engine 5.0 Documentation). Nanite can reduce performance issues by rendering individual pixels instead of in-game triangles if the former requires less processing time.

²⁰ These terms refer to lighting-specific operations that adjusted how the game's textures, special effects, and lighting interact within the game, which can potentially reduce the game's workload and thus improve the game's performance (Areej). These optimization measures were handled mostly by Jordan Tolbert, one of our 3D artists.

any computers using 30 Series GPUs²¹ struggled with the game’s performance the most. Since the 30 Series GPUs utilize a new generation of raytracing technology, our performance issues may have stemmed from any developments with raytracing-heavy calculations. Additionally, our developers discovered that lumen lights, which are used prolifically with the new lighting additions, is a “more optimized, hybrid form” (Areej) of ray-tracing lights used in Unreal Engine 5. Lumen lights are a new addition to the Unreal Engine 5 documentation, and we struggled to utilize them within our scope, so we removed lumen lights and leaned into an older lighting technique, light baking²², to improve the game’s performance.

Unfortunately, this change was not an immediate compromise between our lighting and performance, and the resulting lighting looked muddy and broken on some of the in-game textures (Figure 10). However, our performance and framerates vastly improved after implementing this change, and we prioritized improving the performance since we exhausted our research and optimization options as a team. Thus, we remedied our performance issues by removing lumen lights, and we aimed to improve our lighting with non-lumen methods in future developments.

²¹ GPUs, aka Graphic Processing Units, handle several important calculations and rendering tasks needed in running video games. At the time of this thesis being written, the 30 Series is one of the most up-to-date generations of GPUs with top-grade performance and quality (Intel).

²² Baking lights is the process of calculating lighting effects and storing it in a separate lightmap texture, which means lights and shadows are not dynamically moving. This optimizes gameplay performance by avoiding lighting calculations during runtime (Arm Developer).



Figure 10: Simpler, high-performance lighting with lumen disabled²³

The key takeaway of this pitfall is to communicate and collaborate when it comes to these large, research-heavy issues. Our lighting team initially consisted of two developers new to lighting developments, so collaborating on a solution with other developers expedited the process of researching and tackling the performance bugs. Collaboration and communication are game-changing tools for a game development team for narrowing down possible solutions to a difficult pitfall.

Iterative Lighting Adjustments

After fixing the major performance bugs, our developers continued traversing this pitfall by refining the remaining lighting issues. The previous solution changed the lighting drastically and affected several materials and textures. This created extremely bright and dark areas in the game (Figure 11), which made some texts unreadable and cloaked entire platforming areas into darkness. Additionally, the lighting lost some of its details and depth after this switch, so we needed to pivot our approach to recreate magical, useful lighting effects.

²³ The high-resolution screenshots and lighting fixes were handled by Jordan Tolbert.



Figure 11: Extreme lighting effects following the baking procedure

Like the previous section, we recognized this pitfall by the sheer amount of work and experimenting that our developers invested. Developers constantly added and removed lights depending on their environmental effects, and the entire scene needed to be baked again each time as well, which is a taxing operation considering the size of our level. Developers' computers would sometimes crash or fail to load levels, hampering the time spent on development. The game was to reach its early-access phase within a month, so we needed to fix all the instances of broken lighting in a reasonable amount of time.

Our solution to this pitfall involved collaboration, playtesting, and consistently adjusting lighting through incremental changes. Frequent playtests, with both developers and outside playtesters alike, diagnosed specific issues of lighting being too intense or dim in specific areas, and it informed the lighting team on which areas were critical enough for their focus. Additionally, our original lighting team worked hard into re-implementing lights such that they carefully curated the magical atmosphere again without the help of lumen-related lighting. Our team adopted a trial-and-error method of testing new lighting placements and material adjustments, which incrementally brought the lighting closer to that magical atmosphere that lumen lights previously retained. We also had the help of

engineers with more powerful PC setups for baking the lights again, which circumvented the crashing and lagging problems of light-baking for other developers. By collaborating with the resources and knowledge we had, we slowly and surely remedied the lighting in the level while maintaining high performance rates (Figure 12).



Figure 12: Updated lighting after iterative development

The key takeaway from our lighting experiences is to allow enough time for playtesting and iteration. When it comes to escaping pitfalls, we cannot always guarantee reframing the problem will reveal a simpler solution that solves every bug. Even if another solution would vastly improve the situation, scope management and deadlines still exist, so sometimes iterating on top of existing development is safer than pivoting the approach to development. By planning enough time to review and revise game features, playtesting identifies important bugs and iterating on top of previous development enhances the game's experience.

DISCUSSION

Each game development project will have its own pipeline, work environment, culture, pitfalls, and other one-of-a-kind aspects to their workflow. Not every game has a climbing mechanic or level-related malfunctions, but these situations have general lessons applicable to any game developer and their future projects. By analyzing the engineering pitfalls encountered in Whisker Witchery, we illustrate some key takeaways that are transferrable to any game development project. Treading beyond the scope of Whisker Witchery, this section discusses how these takeaways contribute to the game development industry and help game developers navigate different pitfalls during their development.

Playtesting

Playtesting is a fundamental process to creating a game, and this applies to all stages of a game's development²⁴. Games are meant to be played by people, and by having lots of people play games and feedback, they are actively improving and polishing the player's experience. Reoccurring players, brand-new players, the developers, and more all have their own benefits to playtesting a game because they bring unique feedback to their teams. With Whisker Witchery's development, our team stayed on its own firm playtesting schedule, and it helped our team tremendously with setting up our development's priorities and noticing if we were trapped in a pitfall.

²⁴ Playtesting brings unique benefits at every stage of game development. Playtesting in the earlier stages helps to identify pain points and barriers, and playtesting in later stages helps with finding detrimental bugs before the game's final release. There are several types of playtesting to enhance the user experience and gameplay (80 Level Research Team).

Playtesting is a coveted and crucial practice among all game developers. The feedback and collaboration “provide gamers opportunities to explore the mechanics, levels, and ideas, and they help developers to make games in a shorter time with much lower costs” (80 Level Research Team). The act of playtesting expedites the development process and, when playtesters express the same criticism, guides game developers out of any developmental pitfalls they might be in. The Whisker Witchery team experienced this while developing climbing mechanics, which helped us in identifying and fixing the awkward movement of our mechanics. Playtesting is an essential game development tool for figuring out priorities and urgent tasks within a limited scope. Developers may have their own ideas about the issues in their game, but playtesting illustrates the most relevant issues and will guide developers to the completion of the game’s development.

Additionally, developers have a limited perspective on their game’s experiences, so expanding the amount of people that play and review a game minimizes the overlooked issues. After working on a creative project for long enough, “it becomes very difficult to objectively look at the game from the player’s perspective” (Eng), so having several people play and review a game locates any problems and pitfalls that the developers otherwise would miss.

Revision Control

Version control is an essential tool during game production. Version control allows developers to collaborate and integrate work, experiment in separate branches of production, and access a concise history of updates that can be reverted to at any point (Ernst). When working on a massive large-scale project, being able to reference previous

versions of code or work simultaneously on the project at the same time is largely beneficial to navigating around certain types of pitfalls.

With Whisker Witchery, we encountered a massive series of bugs that threatened the playability of our game. Version control narrowed down these problems to their origin and allowed us to revert comfortably to previous versions as needed, which saved a ton of time and resources on aimlessly debugging. Version control protected the integrity of our product, because the committed history allowed us to start over from a recent point of development instead of running in circles or restarting any code from scratch.

By utilizing version control, game productions are protected from especially challenging, game-breaking pitfalls without compromising too many resources. Thoroughly understanding the tools and advantages provided by any given version control system can save developers hours of wasted time and headaches over nonsensical bugs.

Collaboration

Game productions are massive collaboration projects that require a sufficient level of organization and communication. “Collaboration as teams [is] not only as a strategic practice to overcome various challenges in indie game development but also a core value of the indie culture: teamwork represents unity, openness, sharing, and mutual support” (Freeman & McNeese). By leaning into these social practices, a game’s momentum can soar even within the uncertainty of pitfalls.

Whisker Witchery experienced this with the lighting-related performance issues. By communicating the pitfall to the entire game development team and allowing others to collaborate, the rest of the team contributed additional help and knowledge to the initial

lighting team. Dozens of people combining a bit of knowledge and effort is stronger than two people spending hours spinning circles to try and find new solutions.

Despite the skills of any individual developer, their abilities are limited without interacting with the rest of the game development team. Communication and collaboration “pools scattered resources and link remote individuals, making alternative production and distribution structures for game development possible” (Freeman & McNeese), which can help teams escape dire situations. Each developer brings their own skills, experiences, knowledge, and expertise to the table, so a developer asking for help about an unfamiliar area can be fixed quickly by another developer with thorough knowledge in that field. By asking for clarification and seeking help from other people, game production keeps a good momentum even when facing a challenging pitfall. Thus, collaborating and communicating is expected and necessary to overcome pitfalls in game development.

Iterative Development

“If you think your design will be ‘right/good/complete/perfect’ after just one cycle of creation, you’ll be disappointed and frustrated” (Matson). In terms of design, iteration is the process of refining a creative endeavor in order to create a better version of itself. The iterative process polishes various aspects of game design such as core mechanics and level designs.

With Whisker Witchery’s first iteration of the climbing mechanic, implementing the concepts in-engine led to game-breaking issues, and we continuously reiterated on top of the initial concept to improve it further. Our climbing mechanic started as vine stalks with untraversable topography, which eventually became a climbable tile of overgrown

vines, and we finalized the mechanic with smoother mechanics and enhanced graphics. Since we implemented the concept in-engine as quickly as we did, we budgeted enough time to develop our core mechanic into a fun and beautiful feature for our game. When developers implement their ideas in-engine as soon as they can, they allow enough time to create better, grander, more fun iterations of the concept.

Similarly, the iterative process carried over to our lighting design as well. Our very first lighting instance involved golden specks drifting throughout the level, and we iterated by adding potion glows, moonlight, other atmospheric lights, and further reiterated to adjust baking-related lighting. By implementing concepts quickly and attempting new ideas and processes as early as possible, game production will provide enough time to recover from any unexpected bugs and pitfalls they may encounter.

Pitfalls, the cycle of wasting time and resources, are not always preventable. Scope management and budgeting can considerably reduce the possible pitfalls, but perfectly predicting mishaps in the game development process is impossible. By practicing the iterative process of design, a game production can scope enough time to properly develop their game through these pitfalls and succeed.

CONCLUSION

Pitfalls in game development can be a source of fear and uncertainty for everyone involved. They are problematic approaches that sneak up on a developer and trap them in a cycle of wasting time and resources. Even with a thoughtful production team and thorough organization, production pitfalls can occur sporadically in every game production, and developers must be mindful of these issues to prevent wasting too many resources.

Whisker Witchery overcame their own pitfalls by practicing iterative development, conducting frequent playtests, utilizing revision control, and ensuring consistent collaboration and communication, which are all beneficial tools for game development teams. These tools are essential to any type of game developer, helping them tackle their own obstacles and optimize their resources during game development.

By understanding the importance of these game development principles and practicing them in other projects, developers can overcome similar pitfalls in their own game productions and persevere through the challenges that await them.

REFERENCES

- 02DCE. “Software Engineering: Debugging.” *GeeksforGeeks*, GeeksforGeeks, 18 Apr. 2023, <https://www.geeksforgeeks.org/software-engineering-debugging/>.
- 80 Level Research Team. “Playtesting: Why It Is Worth Every Penny.” *80lv*, 80lv, 2022, <https://80.lv/articles/playtesting-why-it-is-worth-every-penny/>.
- Areej. “Unreal Engine 5 Lumen vs Ray Tracing: Which One Is Better?” *Hardware Times*, 20 Apr. 2022, <https://www.hardwaretimes.com/unreal-engine-5-lumen-vs-ray-tracing-which-one-is-better/>.
- Draven, Derek. “The Biggest Flops from 10 Major Video Game Developers.” *ScreenRant*, 8 Dec. 2021, <https://screenrant.com/biggest-video-game-flops-major-developers/>.
- Eng, Dave. “Play Testing for Success.” *Medium*, Medium, 5 May 2020, <https://medium.com/@davengdesign/play-testing-for-success-5f949356b2fe/>.
- “Entertainment Arts & Engineering.” *Entertainment Arts Engineering*, University of Utah, <https://games.utah.edu/>.
- Ernst, Michael. *Version Control Concepts and Best Practices*, Sept. 2012, https://homes.cs.washington.edu/~mernst/advice/version-control.html#Introduction_to_version_control.
- “Frames per Second.” Edited by Margaret Rouse, *Techopedia*, 14 Mar. 2022, <https://www.techopedia.com/definition/7297/frames-per-second-fps>.
- Freeman, Guo, and Nathan J. McNeese. “Exploring Indie Game Development: Team Practices and Social Experiences in a Creativity-Centric Technology Community.” *Computer Supported Cooperative Work (CSCW)*, vol. 28, no. 3-4, 2019, pp. 723–748., <https://doi.org/10.1007/s10606-019-09348-x>.
- Klappenbach, Michael. “How to Optimize and Improve Graphics Performance and Frame Rates.” Edited by Kayla Dube, *Lifewire*, Lifewire, 5 Nov. 2022, <https://www.lifewire.com/optimizing-video-game-frame-rates-811784>.
- Mattson, Chris. “Iteration: The Most Important Concept in Design.” *The BYU Design Review*, The BYU Design Review, 1 July 2021, <https://www.designreview.byu.edu/collections/iteration-the-most-important-concept-in-design>.

- Mystic Moon Studios. "Whisker Witchery." *Whisker Witchery*, 20 Apr. 2023, https://store.steampowered.com/app/2376280/Whisker_Witchery/.
- O'Connor, Scott W. "How to Create a Project Scope Management Plan." *Northeastern University Graduate Programs*, 8 Dec. 2020, <https://www.northeastern.edu/graduate/blog/scope-management-plan/>.
- "Real-Time 3D Art Best Practices - Lighting." *Arm Guide for Unity Developers*, Arm Developer, <https://developer.arm.com/documentation/102109/0100/Bake-as-much-as-possible>.
- Unreal Engine 5.0 Documentation. "Nanite Virtualized Geometry." *Nanite Virtualized Geometry*, Epic Games, Inc., 2023, <https://docs.unrealengine.com/5.0/en-US/nanite-virtualized-geometry-in-unreal-engine/>.
- "What Is a GPU? Graphics Processing Units Defined." *Intel*, <https://www.intel.com/content/www/us/en/products/docs/processors/what-is-a-gpu.html>.
- Zimmerman, Eric. *Play as Research: The Iterative Design Process*. https://static1.squarespace.com/static/579b8aa26b8f5b8f49605c96/t/59921253cd39c3da5bd27a6f/1502745178453/Iterative_Design.pdf, Accessed 12 Apr. 2023.

Name of Candidate: Ray Parker

Date of Submission: May 1, 2023